

Once Generated, Many Times Served: Characterizing and Exploiting Content Recycling in Deep Research Agent Inference

Anonymous submission

Abstract

Deep research agents—multi-agent systems that search, read, and write long, citation-grounded reports—are bottlenecked by LLM inference. We present the first content-level characterization of deep research inference at the LLM-call layer (800 tasks, 15,296 calls). We found a defining property: **content recycling**. Content is generated once, yet later calls serve it again and again. They re-read it during prefill and re-generate it during decode. Recycling is pervasive but heterogeneous: some calls copy their inputs almost verbatim; others rewrite and condense (copy rates 8–91%, bimodal). **Once generated, many times served**. Existing techniques cannot collect this waste: (1) cross-request KV reuse perturbs downstream decisions in multi-agent pipelines and degrades report quality; (2) uniform speculative decoding wins on copy-heavy calls but loses on rewrite-heavy ones. We exploit recycling on both sides of inference. On the prefill side, we reuse upstream KV caches and preserve downstream decisions via attention-guided selective recomputation. On the decode side, we route speculative decoding by predicted copy rate: the prediction selects the drafter and sets the draft depth. On mainstream deep research benchmarks, our system cuts end-to-end latency by up to 56.3% and TTFT by [TBD]% versus vLLM. Report quality is statistically unchanged.

Introduction

Deep research agents turn a question into an investigation. A supervisor first decomposes the question into sub-tasks and dispatches them to several researchers. Each researcher searches the web, reads sources, and writes a *summary*. The supervisor reflects on the returned summaries and may launch further rounds of research. A writer finally reads all summaries and produces a long, citation-grounded *report*. Every role issues one or even more LLM calls. In this pipeline the bottleneck is LLM inference, not retrieval: on the workload we study, the LLM calls take about 92% of end-to-end wall-clock time. We study this workload at the LLM-call layer, over a trace of 800 tasks and 15,296 generation calls, and find a defining property: **content recycling**. A span of text is generated once, yet the pipeline serves it many times. This recycling appears on both sides of inference. During

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

prefill, the same content is re-read by later calls, so its KV cache is recomputed from scratch again and again. During decode, the content is re-generated—copied, often verbatim, into later outputs. To our knowledge, this is the first content-level characterization of deep research inference; prior agent traces stop at the systems level and discard the text (Zhu et al. 2026). *Once generated, many times served*.

Recycling is easy to name but hard to collect, because the workload is heterogeneous while existing techniques are uniform. On the prefill side, cross-request KV reuse skips recomputation by blending a cached KV into the new prompt (Yao et al. 2025; Ye et al. 2025; Liu et al. 2025). But the blended KV is only approximate. Blending it blindly shifts what a downstream agent attends to, which changes the agent’s decisions and degrades report quality. Prior methods choose which tokens to recompute by KV deviation alone (Yao et al. 2025); yet a token whose KV deviates greatly but that no downstream query attends to does not affect the output. On the decode side, suffix-based speculation drafts the next tokens directly from repeated content (Oliaro et al. 2025), but it runs the same way on every call. Copy rates vary widely: some calls copy their input almost verbatim, others rewrite and condense it (8–91%, bimodal). A single fixed drafter therefore wins on copy-heavy calls but loses on rewrite-heavy ones. Prior methods adapt the drafter *during* a call, tuning draft length from the drafts accepted so far (Huang, Guo, and Wang 2025; Hou et al. 2025). None routes on a call’s copy rate—the property that decides which drafter fits—which our characterization shows is predictable *before* the call begins.

We exploit recycling on both sides, and we match the mechanism to each call. On the prefill side, we propose *attention-guided selective recomputation*: we recompute a reused token only when its KV deviation is large *and* the downstream query attends to it heavily; a token that deviates greatly but draws little attention is reused unchanged. On the decode side, we route speculative decoding by predicted copy rate: a lightweight predictor reads the target model’s hidden state and, under a simple cost model, selects both the drafter and the draft depth, so that each call runs its most economical strategy. For rewrite-heavy calls, where little text can be copied, off-the-shelf draft heads fail on this workload; we therefore train an EAGLE-3 draft head on the agent’s own trajectories. We train both the copy-rate predictor and the specialized head. On mainstream deep research bench-

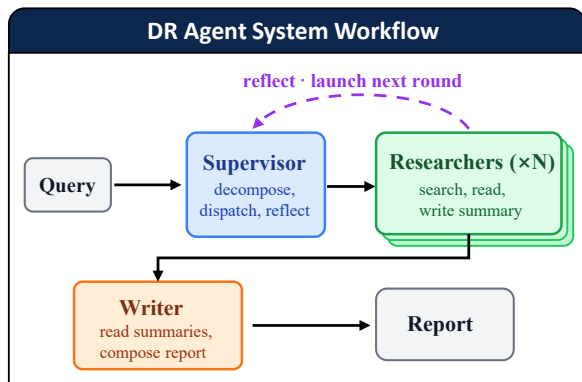


Figure 1: The deep research agent workflow. A supervisor decomposes the query and dispatches sub-tasks to parallel researchers; each researcher searches, reads, and writes a summary; the supervisor reflects and may launch further rounds; a writer composes the final report.

marks, our system cuts end-to-end latency by up to 56.3% versus vLLM, with report quality statistically unchanged. This paper makes four contributions:

- **Characterization.** We present the first content-level characterization of deep research inference at the LLM-call layer, and show that content is recycled on both the prefill and the decode side, in amounts that vary sharply across calls.
- **Attention-guided selective recomputation.** On the prefill side, we recompute a reused token only when its KV deviation and its downstream attention are *both* high, avoiding both the quality loss of blind blending and the wasted work of deviation-only selection.
- **Copy-rate-routed speculative decoding.** On the decode side, a hidden-state predictor selects the drafter and the draft depth per call under a cost model; for rewrite-heavy calls we add an EAGLE-3 head specialized on the agent’s own trajectories.
- **System and evaluation.** We integrate both sides and evaluate end-to-end on deep research benchmarks, cutting latency by up to 56.3% with statistically unchanged report quality.

Related Work

Agent workload studies. Serving-oriented studies of agent workloads are recent. TraceLab characterizes coding-agent traces at the systems level and discards message text during anonymization, so content overlap cannot be recovered from its data (Zhu et al. 2026). Similar systems-level studies cover ReAct-style agents (Yuan et al. 2026) and memory-augmented agents (Omri et al. 2026). Content overlap has been measured, but at other layers: term adoption in agentic search-query logs (Ning et al. 2026), redundancy of final reports under LLM judges (Fan et al. 2025), and byte-exact duplication inside single-turn RAG prompts (Schelpe

2026). Our copy metrics descend from extractiveness measures in summarization (Grusky, Naaman, and Artzi 2018; Long et al. 2026). None of these reaches the LLM-call layer of a multi-agent pipeline with the text intact, which is where content recycling lives and where we measure it.

KV cache reuse. Prefix caching reuses the KV of an identical prompt prefix (Zheng et al. 2024). A writer reads its summaries in arbitrary order and subsets, so a shared prefix rarely exists; agent-aware systems work around this by fixing the prefix order (Jin et al. 2025). Position-independent reuse lifts the restriction: it blends cached KV into a new prompt and selectively recomputes a few tokens. CacheBlend picks those tokens by KV deviation (Yao et al. 2025); KVCMM aligns caches across agents with anchors (Ye et al. 2025); DroidSpeak shares caches across models (Liu et al. 2025); RelayCaching forwards decode-produced KV to downstream prefill (Geng et al. 2026). All of them decide what to recompute from the cache side alone. None asks what the downstream call will attend to, and a token can deviate greatly yet draw no attention. Our criterion is the product of deviation and downstream attention, derived from a first-order bound on the output error.

Speculative decoding. Retrieval-based drafters propose tokens copied from a reference or from the context (Yang et al. 2023; Oliaro et al. 2025). Trained draft heads predict tokens from the target’s hidden state (Li et al. 2025). Adaptive control exists, but it acts within a call: SpecDec++ tunes the draft length from the drafts accepted so far (Huang, Guo, and Wang 2025), and BanditSpec selects speculation hyperparameters online (Hou et al. 2025). Drafters have also been trained toward a target distribution, offline or on live traffic (Zhou et al. 2024; Liu et al. 2024; Wang et al. 2026). Agent deployments so far use heads as-is: a production commerce agent runs EAGLE-3 training-free and leaves domain fine-tuning to future work (Qin et al. 2026). We differ on both counts. We route each call between a retrieval drafter and a trained head, using a copy rate predicted before generation begins, and we specialize the head on the agent’s own trajectories.

Agent serving systems. Parrot exposes inter-request structure to the engine through semantic variables (Lin et al. 2024), and Autellix schedules agent programs to reduce end-to-end latency (Luo et al. 2026). These systems act at the scheduling layer: they decide when and where calls run. We act below them, cutting the cost of each call itself, and the two layers compose.

Content Recycling in Deep Research

We instrument a production-shaped deep research pipeline and record, for every LLM call, its full input, its full output, and its decode timing; our trace spans 800 tasks and 15,296 generation calls. The pipeline moves information as text: a researcher’s summary is written once, then handed to the supervisor and to the writer. Following that content through the trace, we find it is not consumed once but served many times, and the waste lands on both stages of inference—prefill and decode.

Recycling on the prefill side. The same content is read again and again. The supervisor re-reads the accumulated summaries each round to decide the next step, and the writer reads all of them to compose the report. Every such call pre-fills those summaries from scratch, recomputing a KV cache that the producing call had already computed. The writer’s prompt is essentially the concatenation of all summaries—tens of thousands of tokens, almost none of them new—yet its KV is built anew on every request. This is repeated encoding of content the pipeline has already encoded.

Recycling on the decode side. The same content is generated again. Calls copy their input into their output rather than composing from scratch: across calls the copy rate ranges from 8% to 91% and is bimodal—some calls transcribe their input nearly verbatim, others rewrite and condense it. Every copied token is decoded anew, one target forward pass at a time, although it already exists in the input. The two sides are the same waste seen twice: content is generated once, then read and regenerated many times.

System Design

Our system serves the deep research pipeline on a single inference engine and attacks content recycling on both sides of a call. When a call re-reads content that an earlier call produced, we reuse that content’s KV cache and recompute only the tokens that matter (Section). When a call generates, we speculate, choosing the drafter and the draft depth that fit how much it copies (Section). Both mechanisms ask the same question—how will this call use its content?—and both read a signal that is available before the call finishes: the downstream attention over the reused tokens, and the call’s copy rate.

Attention-Guided Selective Recomputation

A downstream call—a supervisor reaching a decision, or the writer composing the report—re-reads summaries that researchers have already generated. Recomputing their KV cache during prefill repeats work the pipeline already did. We instead load the cached KV. That KV was computed inside the producing call, at different positions and in a different context, so reusing it verbatim is inaccurate; blended into the new prompt blindly, it shifts the downstream output.

The design question is *which* reused tokens to recompute under a fixed budget. Prior work selects them by KV *deviation*: recompute the tokens whose cached value differs most from a freshly computed one (Yao et al. 2025). We show that deviation alone spends the budget in the wrong place.

Consider one attention head at a downstream query position q , attending over the reused tokens. Its output is $o = \sum_j a_j v_j$, with attention weights $a_j = \text{softmax}(q \cdot k_j)$ and values v_j . Reuse replaces v_j by $\hat{v}_j = v_j + \Delta v_j$. To first order, holding the weights fixed, the output error is

$$\Delta o = \sum_j a_j \Delta v_j. \quad (1)$$

Recomputing a subset S of tokens sets $\Delta v_j = 0$ for $j \in S$. By the triangle inequality the residual error is bounded by

Algorithm 1: Attention-guided selective recomputation for one call

Require: cached KV $\{\hat{k}_j, \hat{v}_j\}$ of the reused tokens; budget B

- 1: recompute the gate layer; score each token’s deviation $\|\Delta v_j\|$
- 2: read the end-of-prompt query’s attention a_j at the gate layer
- 3: $S \leftarrow$ the B tokens with the largest $a_j \|\Delta v_j\|$
- 4: recompute the KV of S at all layers; keep the rest cached
- 5: continue prefill with the blended KV

$\sum_{j \notin S} a_j \|\Delta v_j\|$, which is separable across tokens; minimizing this bound under $|S| \leq B$ recomputes the tokens with the largest $a_j \|\Delta v_j\|$. The criterion is a *product*: a token must both deviate *and* be attended to. A token whose value is badly wrong but that no downstream query reads ($a_j \approx 0$) does not change the output and is left reused, whereas deviation alone would waste the budget on it. Recomputing exactly the tokens that move the output is what preserves the downstream decision that blind reuse would otherwise perturb.

We obtain both signals inside the prefill we already run. For $\|\Delta v_j\|$ we recompute the reused tokens at a single gate layer and compare to the cached value, which predicts the deviation at the remaining layers. For a_j we read the attention that the end-of-prompt query—the position about to drive the call’s generation—places on the reused tokens at that same layer. This query runs on the reused, still-approximate keys, so a_j is a one-pass proxy for the true downstream attention; we accept it because it needs no extra forward pass and its error is second order in the key deviations we already treat as small. Algorithm 1 summarizes the procedure; only the scoring in line 3 differs from deviation-only reuse.

Copy-Rate-Routed Speculative Decoding

On the decode side we speculate: a cheap drafter proposes several next tokens, the target verifies them in a single forward pass, and correct guesses are accepted with no change to the output. The gain depends on the drafter fitting the call. A *suffix* drafter proposes tokens copied from the call’s own context (Oliaro et al. 2025); it excels when the call copies its input and stalls when the call rewrites it. A trained *draft head* (EAGLE-3 (Li et al. 2025)) predicts tokens from the target’s hidden state; it needs no copied text and covers the rewrite regime. Neither drafter is best for both, and our characterization shows the two regimes are common and sharply separated.

We route each call by its copy rate. A lightweight predictor f reads the target’s hidden state at the start of the call and estimates the copy rate $\hat{c}$; a high $\hat{c}$ sends the call to the suffix drafter, a low $\hat{c}$ to the specialized head. We realize this in a single engine that hosts both drafters over one target model, and each request selects its drafter. Hosting both in one engine costs far less memory than running one server per drafter: the suffix drafter has no parameters and the head adds a few gigabytes, rather than duplicating the target.

The draft *depth*—how many tokens to propose—is a bud-

get question. A drafted slot costs a fixed per-step overhead plus a per-slot cost, and it pays off only when its acceptance probability clears a break-even rate. We measure the per-slot cost at 0.62 ms and the resulting break-even acceptance at 2.4%. We set the depth per call from $\hat{c}$: a high-copy call, whose deep suffixes stay above break-even, drafts deep; a low-copy call drafts shallow or not at all, so a rewriting call—or a misprediction—never pays a large speculation tax. This saving is largest under concurrency, where wasted drafts contend for the compute of every other request.

Training the Learned Components

Copy-rate predictor. We label every call in our trace with its measured copy rate and train f to regress it from the target’s hidden state at the call’s first decoded positions. f is a small head over a hidden state the target already produces, so it adds negligible latency.

Specialized draft head. Off-the-shelf EAGLE-3 heads are trained on generic chat data and fail on this workload: their acceptance is low, and the failure tracks the deep-research writing style rather than language. We therefore train the head on the agent’s own trajectories—the prompts and outputs the target itself produced while running the pipeline, which are on-policy by construction. We report the trained head’s acceptance in the evaluation.

Experiments

Conclusion

References

- Fan, T.; Niu, X.; Zheng, Y.; Zhang, F.; Huang, C.; Chen, B.; Lin, J.; and Huang, C. 2025. Understanding DeepResearch via Reports. arXiv:2510.07861.
- Geng, Y.; Gao, Y.; Wu, W.; Liu, G.; and Liu, J. 2026. RelayCaching: Accelerating LLM Collaboration via Decoding KV Cache Reuse. arXiv:2603.13289.
- Grusky, M.; Naaman, M.; and Artzi, Y. 2018. Newsroom: A Dataset of 1.3 Million Summaries with Diverse Extractive Strategies. In *NAACL-HLT*.
- Hou, Y.; Zhang, F.; Du, C.; Zhang, X.; Pan, J.; Pang, T.; Du, C.; Tan, V. Y. F.; and Yang, Z. 2025. BanditSpec: Adaptive Speculative Decoding via Bandit Algorithms. arXiv:2505.15141.
- Huang, K.; Guo, X.; and Wang, M. 2025. SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths. arXiv:2405.19715.
- Jin, S.; Liu, X.; Wu, Y.; Zheng, H.; Zhang, Q.; Prakash, A.; Lentz, M.; Zhuo, D.; Qian, F.; and Mao, Z. M. 2025. Plato: Plan to Efficiently Decode for Large Language Model Inference. arXiv:2402.12280.
- Li, Y.; Wei, F.; Zhang, C.; and Zhang, H. 2025. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. arXiv:2503.01840.
- Lin, C.; Han, Z.; Zhang, C.; Yang, Y.; Yang, F.; Chen, C.; and Qiu, L. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 929–945. Santa Clara, CA: USENIX Association. ISBN 978-1-939133-40-3.
- Liu, X.; Hu, L.; Bailis, P.; Cheung, A.; Deng, Z.; Stoica, I.; and Zhang, H. 2024. Online Speculative Decoding.
- Liu, Y.; Huang, Y.; Yao, J.; Feng, S.; Gu, Z.; Du, K.; Li, H.; Cheng, Y.; Jiang, J.; Lu, S.; Musuvathi, M.; and Choukse, E. 2025. DroidSpeak: KV Cache Sharing for Cross-LLM Communication and Multi-LLM Serving. arXiv:2411.02820.
- Long, Y.; Wu, X.; Zhang, Y.; Wen, X.; Zhou, Y.; and Hong, S. 2026. Copy-Paste to Mitigate Large Language Model Hallucinations. arXiv:2510.00508.
- Luo, M.; Shi, X.; Cai, C.; Zhang, T.; Wong, J.; Wang, Y.; Wang, C.; Huang, Y.; Chen, Z.; Gonzalez, J. E.; and Stoica, I. 2026. Agentix: An Efficient Serving Engine for LLM Agents as General Programs.
- Ning, J.; Coelho, J.; Kong, Y.; Long, Y.; Martins, B.; Magalhães, J.; Callan, J.; and Xiong, C. 2026. Agentic Search in the Wild: Intents and Trajectory Dynamics from 14M+ Real Search Requests.
- Oliaro, G.; Jia, Z.; Campos, D.; and Qiao, A. 2025. SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications. arXiv:2411.04975.
- Omri, Y.; Gan, Z.; Broveak, Z.; Geens, R.; He, Z.; Pentland, A.; Verhelst, M.; Weissman, T.; and Tambe, T. 2026. Agent Memory: Characterization and System Implications of Stateful Long-Horizon Workloads. arXiv:2606.06448.
- Qin, A.; Wan, J.; Mudunuri, S.; and Manoharan, S. 2026. Accelerating PayPal’s Commerce Agent with Speculative Decoding: An Empirical Study on EAGLE3 with Fine-Tuned Nemotron Models. arXiv:2604.19767.
- Schelppe, S. 2026. Byte-Exact Deduplication in Retrieval-Augmented Generation: A Three-Regime Empirical Analysis Across Public Benchmarks. arXiv:2605.09611.
- Wang, J.; Bie, F.; Li, J.; Zhou, Z.; Shao, Z.; Wang, Y.; Liu, Y.; Wu, Q.; May, A.; Yanamandra, S.; Zhang, C.; Dao, T.; Liang, P.; Athiwaratkun, B.; Song, S. L.; Xu, C.; and Wu, X. 2026. When RL Meets Adaptive Speculative Training: A Unified Training-Serving System. arXiv:2602.06932.
- Yang, N.; Ge, T.; Wang, L.; Jiao, B.; Jiang, D.; Yang, L.; Majumder, R.; and Wei, F. 2023. Inference with Reference: Lossless Acceleration of Large Language Models. arXiv:2304.04487.
- Yao, J.; Li, H.; Liu, Y.; Ray, S.; Cheng, Y.; Zhang, Q.; Du, K.; Lu, S.; and Jiang, J. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion.
- Ye, H.; Gao, Z.; Ma, M.; Wang, Q.; Fu, Y.; Chung, M.-Y.; Lin, Y.; Liu, Z.; Zhang, J.; Zhuo, D.; and Chen, Y. 2025. KVCOMM: Online Cross-context KV-cache Communication for Efficient LLM-based Multi-agent Systems. arXiv:2510.12872.
- Yuan, Y.; Nayak, A.; Kundu, S.; and Talati, N. 2026. Agentic AI Workload Characteristics. arXiv:2605.26297.
- Zheng, L.; Yin, L.; Xie, Z.; Sun, C.; Huang, J.; Yu, C. H.; Cao, S.; Kozyrakis, C.; Stoica, I.; Gonzalez, J. E.; Barrett, C.; and Sheng, Y. 2024. SGLang: Efficient Execution of Structured Language Model Programs.

Zhou, Y.; Lyu, K.; Rawat, A. S.; Menon, A. K.; Ros-tamizadeh, A.; Kumar, S.; Kagy, J.-F.; and Agarwal, R. 2024. DistillSpec: Improving Speculative Decoding via Knowledge Distillation. arXiv:2310.08461.

Zhu, K.; Jacob, M.; Ma, C.; Pan, Y.; Wang, S.; Krishnamurthy, A.; and Kasikci, B. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560.